FINAL REPORT

Group 5

2026-06-11

This is the final report for group 5 which authors are: Alessia Visani, Veronica Boccaletti, Francesca Crispino, Valentina Molinari, Samuele Hamama, Mahsa Samadi, Tommaso Nicolis

aim:analysis of a data set of DNA methylation at genome-wide CpG levels

Pipeline steps

1. Load raw data with minfi and create an object called 'RGset' storing the RGChannelSet object.

First, check whether the `BiocManager` package is installed; if not, install it.

```
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
```

Install the `minfi` package using `BiocManager`, if it's not already available; and after that load `minfi`.

```
#BiocManager::install("minfi")

library(minfi)
library(knitr)
```

Clean the R environment by removing all existing objects and set the working directory, where the raw data are stored.

```
rm(list = ls())

directory = "/Users/aleonmac/Desktop/UNI/DNA_RNA/final_report/Input_Data" # Insert here
your directory
setwd(directory)
```

Load sample sheet using the `read.metharray.sheet()` function which checks that in the directory there is a sample sheet; it also modifies the sample sheet included in the input folder by adding an extra column `Basename`. This new column defines the pattern that the function will use to load the `idat` files in the working environment.

```
baseDir <- (directory)
targets <- read.metharray.sheet(baseDir)
```

```
## [read.metharray.sheet] Found the following CSV files:
```

```
## [1] "/Users/aleonmac/Desktop/UNI/DNA_RNA/final_report/Input_Data/SampleSheet_Report_
II.csv"
```

```
# Verify and print the number of imported samples
paste("Number of found samples: ", nrow(targets))
```

```
## [1] "Number of found samples:  8"
```

Read the data using the function `read.metharray.exp()` and save it in the current working directory.

```
RGset <- read.metharray.exp(targets = targets)
save(RGset, file = "RGset.RData")
```

2. Create the dataframes Red and Green to store the red and green fluorescences respectively.

Use the `getRed()` and `getGreen()` functions from the `minfi` package, to access the red (Cy5) and green (Cy3) signal channels, respectively.
The extracted signals of raw data from an Illumina microarray experiment, are converted into two separate data frames, named 'Red' and 'Green', to facilitate data inspection, manipulation, and analysis.
In each data frame, rows correspond to probes, and columns correspond to samples. To validate the extraction process, use the `dim()` and `head()` functions. `dim()` checks the dimensions of the matrices, while `head()` displays the first few rows.

```
Red <- data.frame(getRed(RGset))
```

```
Green <- data.frame(getGreen(RGset))
```

3. Identify the Red and Green fluorescences for the assigned address.

Create Type I and Type II dataframes.

```
df_I  <- data.frame(getProbeInfo(RGset, type = "I"))
```

```
## Loading required package: IlluminaHumanMethylation450kmanifest
```

```
df_II <- data.frame(getProbeInfo(RGset, type = "II"))
```

Create the objects for the table: check first if the given address is present in at least one of the two columns `AddressA` or `AddressB` (which are the physical array addresses for one probe sequence that targets a specific state).
Then, extract fluorescence intensities from the Type I and Type II manifests for the probe identified by address "47683488".

```
address <- "47683488"

if (address %in% df_I$AddressA | address %in% df_I$AddressB) {
  matched_row <- df_I[df_I$AddressA == address | df_I$AddressB == address, ]
  p_type  <- "Type I"
  p_color <- matched_row$Color
  } else if (address %in% df_II$AddressA) {
  p_type  <- "Type II"
  p_color <- "-"
  }

matched_row <- df_II[df_II$AddressA == address,]
```

Create a report table and populate it by combining the Red and Green fluorescence values for each sample and including the probe type and color.

```
report_table <- data.frame(
  Sample = colnames(Red),
  `Red fluor` = as.numeric(Red[address, ]),
  `Green fluor` = as.numeric(Green[address, ]),
  Type = p_type,
  Color = p_color,
  check.names = FALSE
  )

report_table
```

```
##                          Sample Red fluor Green fluor    Type Color
## 1 GSM5319592_200121140049_R01C02       235        1428 Type II     -
## 2   GSM5319603_3999356129_R02C02       118         334 Type II     -
## 3   GSM5319604_3999547012_R02C01       318        1485 Type II     -
## 4   GSM5319607_3999547012_R03C02       325        1714 Type II     -
## 5   GSM5319609_3999547016_R02C01       248         401 Type II     -
## 6   GSM5319613_3999547017_R02C01       333        2422 Type II     -
## 7   GSM5319615_3999547017_R04C01       517        2592 Type II     -
## 8   GSM5319616_3999547017_R06C01       385        1507 Type II     -
```

There is an `AddressA` with address 47683488, and it is associated to the probe 'cg13691428', Type II. From the `df_II` dataframe we can notice that the column containing the color of the channel is empty, as all the Type II probes will emit in Red or Green depending on the methylation status of the target CpG site. In addition, also `AddressB` is empty, as type II probes only consider `AddressA`.

3.1. Check in the manifest file if the address corresponds to a Type I or a Type II probe and, in case of Type I probe, report its color.

Load the Illumina manifest (*Illumina450Manifest_clean.RData*), in which control probes and SNP-associated probes (those prefixed with "rs") have been removed to avoid ambiguity due to polymorphisms.

```
load(file.path(directory, "Illumina450Manifest_clean.RData"))
```

Check if the address is in AddressA_ID or AddressB_ID.

```
type_A <- Illumina450Manifest_clean[Illumina450Manifest_clean$AddressA_ID == "4768348
8", 'Infinium_Design_Type']
type_B <- Illumina450Manifest_clean[Illumina450Manifest_clean$AddressB_ID == "4768348
8", 'Infinium_Design_Type']

paste("Type from AddressA_ID:",type_A)
```

```
## [1] "Type from AddressA_ID: II"
```

```
paste("Type from AddressB_ID:",type_B)
```

```
## [1] "Type from AddressB_ID: "
```

Since the assigned address corresponds to a Type II probe, we're in a comparative array two channel colors. Therefore, we have no record of the color in the manifest file designated column.

4. Create the object MSet.raw

Convert the `RGChannelSet` to an `MSet` object using `preprocessRaw()`.
This function takes the Red and the Green channel of an Illumina methylation array and converts it into a `MethylSet` containing the methylated and unmethylated signal.

```
MSet.raw <- preprocessRaw(RGset)
MSet.raw
```

```
## class: MethylSet
## dim: 485512 8
## metadata(0):
## assays(2): Meth Unmeth
## rownames(485512): cg00050873 cg00212031 ... ch.22.47579720R
##   ch.22.48274842R
## rowData names(0):
## colnames(8): GSM5319592_200121140049_R01C02
##   GSM5319603_3999356129_R02C02 ... GSM5319615_3999547017_R04C01
##   GSM5319616_3999547017_R06C01
## colData names(7): SampleID Sex ... Basename filenames
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
## Preprocessing
##   Method: Raw (no normalization or bg correction)
##   minfi version: 1.56.0
##   Manifest version: 0.4.0
```

```
save(MSet.raw,file="MSet_raw.RData")
```

Create the objects for methylation and unmenthylation signals.

```
Meth <- as.matrix(getMeth(MSet.raw))
```

```
Unmeth <- as.matrix(getUnmeth(MSet.raw))
```

When moving from an `RGChannelSet` to a `MethylSet` object, the dataset undergoes a fundamental shift in perspective: from physical chip coordinates to biological CpG coordinates. Therefore, we want to find the `cpg_name` for the given address.

```
cpg_name <- df_II$Name[df_II$AddressA == address]
cpg_name
```

```
## [1] "cg13691428"
```

Grab the values for the address in the `MSet`.

```
meth_values   <- as.numeric(Meth[cpg_name, ])
unmeth_values <- as.numeric(Unmeth[cpg_name, ])
```

Now extract the values for the address in the `RGset`.

```
red_values    <- as.numeric(Red[address, ])
green_values  <- as.numeric(Green[address, ])
```

Perform a double check: for Type II, Green MUST be equal to Meth values, and Red MUST equal Unmeth values. Use the function `identical()` and not `==` because we want a boolean output not a comparison between arrays column by column.

```
identical(meth_values, green_values)
```

```
## [1] TRUE
```

```
identical(unmeth_values, red_values)
```

```
## [1] TRUE
```

5. Perform different quality checks and provide a brief comment to each step:

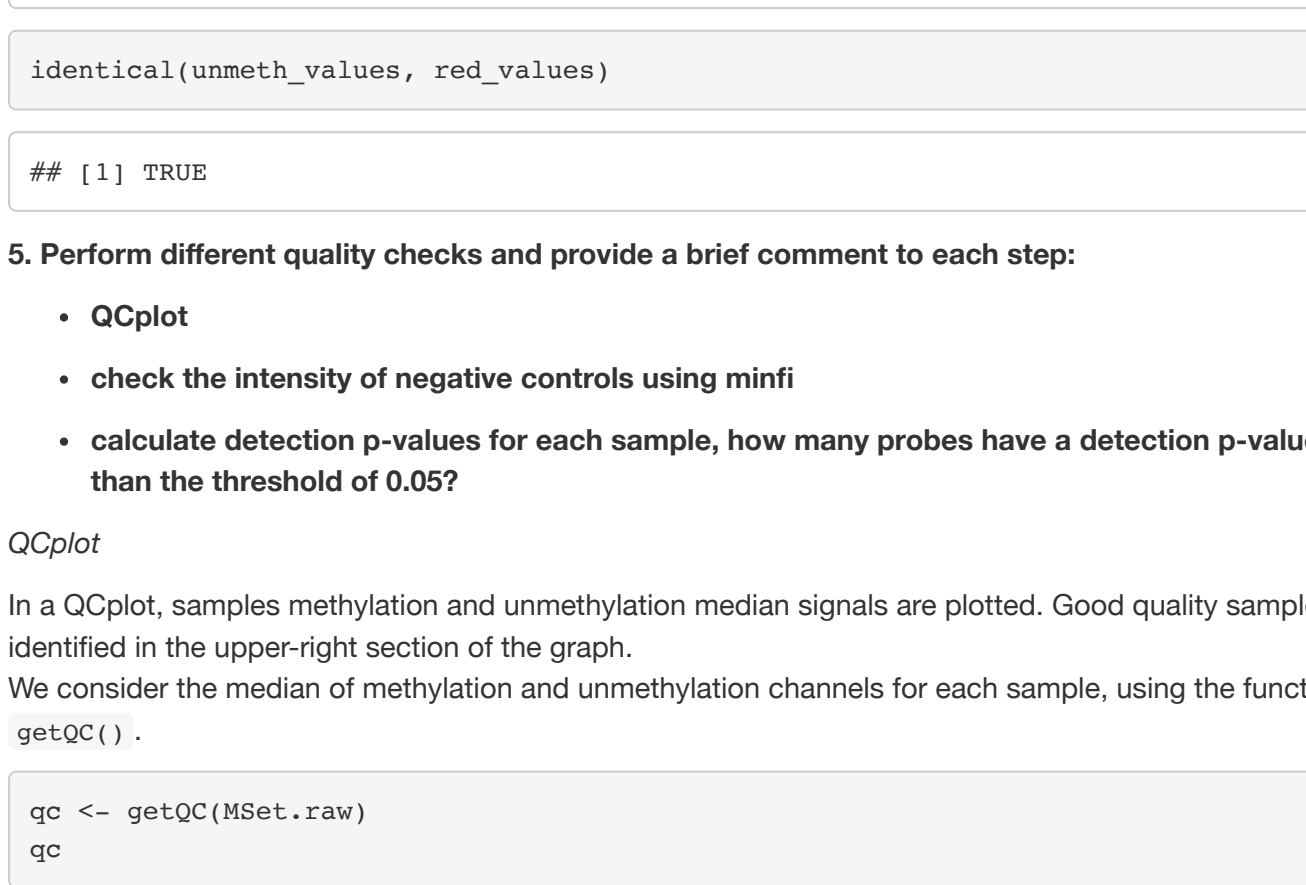
- **QCplot**
- **check the intensity of negative controls using minfi**
- **calculate detection p-values for each sample, how many probes have a detection p-value higher than the threshold of 0.05?**

QCplot

In a QCplot, samples methylation and unmethylation median signals are plotted. Good quality samples are identified in the upper-right section of the graph.
We consider the median of methylation and unmethylation channels for each sample, using the function `getQC()`.

```
qc <- getQC(MSet.raw)
qc
```

```
plotQC(qc)
```



Samples 1, 3, and 8 display minor baseline shifts but remain acceptable. However, Samples 2 and 5 drop significantly lower, reflecting poor hybridization or low DNA input that warrants cautious downstream monitoring.

Check the intensity of negative controls using `minfi`

To know the names and the numbers of the different types of control probes, apply the `**` getProbeInfo() `**` function to the `RGset` object (we evaluate the intensities of the control probes).

```
getProbeInfo(RGset, type = "Control")
```

```
## DataFrame with 848 rows and 4 columns
##         Address        Type       Color  ExtendedType
##     <character> <character> <character>   <character>
## 1      27630314    STAINING         Red    DNP (High)
## 2      43603326    STAINING      Purple     DNP (Bkg)
## 3      41666334    STAINING       Green Biotin (High)
## 4      34648333    STAINING        Blue  Biotin (Bkg)
## 5      63642461   EXTENSION         Red Extension (A)
## ...         ...         ...         ...           ...
## 844    65622344      NORM_T      Purple      Norm_T59
## 845    57640346      NORM_C       Green      Norm_C60
## 846    47715450      NORM_T      Purple      Norm_T60
## 847    28673402      NORM_C       Green      Norm_C61
## 848    13742412      NORM_T      Purple      Norm_T61
```

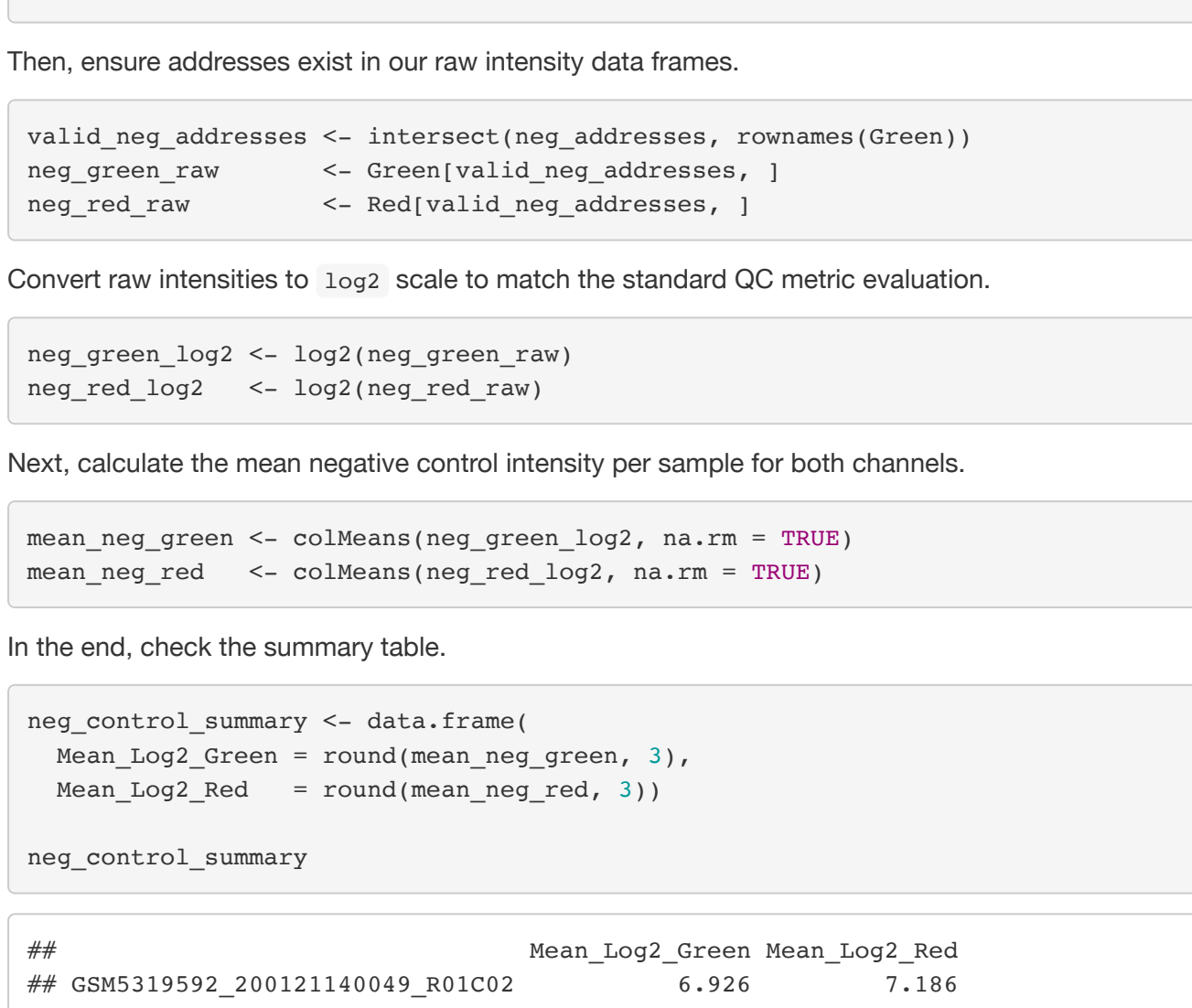
```
df_TypeControl <- data.frame(getProbeInfo(RGset, type = "Control"))
table(df_TypeControl$Type)
```

```
##
##  BISULFITE CONVERSION I BISULFITE CONVERSION II               EXTENSION
##                      12                       4                       4
##           HYBRIDIZATION                NEGATIVE         NON-POLYMORPHIC
##                       3                     613                       4
##                  NORM_A                  NORM_C                  NORM_G
##                      32                      61                      32
##                  NORM_T             RESTORATION           SPECIFICITY I
##                      61                       1                      12
##          SPECIFICITY II                STAINING          TARGET REMOVAL
##                       3                       4                       2
```

From the table we can check the categories of the different control probes: the NEGATIVE controls are much higher in numbers.

Then, use the `controlSripPlot()` function to plot (scatter plot) the intensity values of the controls probes in the samples.

```
controlStripPlot(RGset, controls="NEGATIVE")
```



All the negative controls are all fine, as they are all below 1000 (log2(1000)=10) as expected.

We noticed that there is a minor error in the package: green and red labels are swapped. So, we proceeded to verify the intensities in another manner.
First, address IDs strictly assigned to NEGATIVE controls.

```
neg_addresses <- df_TypeControl$Address[df_TypeControl$Type == "NEGATIVE"]
```

Then, ensure addresses exist in our raw intensity data frames.

```
valid_neg_addresses <- intersect(neg_addresses, rownames(Green))
neg_green_raw       <- Green[valid_neg_addresses, ]
neg_red_raw         <- Red[valid_neg_addresses, ]
```

Convert raw intensities to `log2` scale to match the standard QC metric evaluation.

```
neg_green_log2 <- log2(neg_green_raw)
neg_red_log2   <- log2(neg_red_raw)
```

Next, calculate the mean negative control intensity per sample for both channels.

```
mean_neg_green <- colMeans(neg_green_log2, na.rm = TRUE)
mean_neg_red   <- colMeans(neg_red_log2, na.rm = TRUE)
```

In the end, check the summary table.

```
neg_control_summary <- data.frame(
  Mean_Log2_Green = round(mean_neg_green, 3),
  Mean_Log2_Red   = round(mean_neg_red, 3))

neg_control_summary
```

```
##                                Mean_Log2_Green Mean_Log2_Red
## GSM5319592_200121140049_R01C02           6.926         7.186
## GSM5319603_3999356129_R02C02             6.039         6.244
## GSM5319604_3999547012_R02C01             7.156         7.553
## GSM5319607_3999547012_R03C02             7.350         7.876
## GSM5319609_3999547016_R02C01             6.586         7.262
## GSM5319613_3999547017_R02C01             7.121         7.532
## GSM5319615_3999547017_R04C01             7.380         7.678
## GSM5319616_3999547017_R06C01             7.226         7.674
```

The negative controls are all fine, as they all are below 1000; which is the same as before in the scatter plot.

Calculate detection p-values for each sample, how many probes have a detection p-value higher than the threshold of 0.05?

Use the `detectionP()` function on the `RGset` to obtain the p-value for every genomic position in every sample.

```
threshold <- 0.05
detP <- detectionP(RGset)
```

The `detP` object is a matrix with CpG probes names as rows indexes and sample names in the columns. The lowest the p-value, the better (small p-values indicate a good position).

Filter out the bad samples by considering the condition `>0.05`.

```
failed <- detP>threshold
table(failed)
```

```
## failed
##   FALSE    TRUE
## 3877813    6283
```

Therefore, in total there are 6283 failed (TRUE) probe positions across all 8 samples combined.
Create a summary table and insert in it the name of the samples containing the failed probes, the number of failed positions and the percentage of failed probes.

```
true_counts <- colSums(failed, na.rm = TRUE)

failed_table <- data.frame(
  Sample = colnames(failed),
  `n Failed Positions` = true_counts,
  `% Failed Probes` = (means_of_columns <- colMeans(failed))*100,
  row.names = NULL,
  check.names = FALSE
  )

failed_table
```

```
##                          Sample n Failed Positions % Failed Probes
## 1 GSM5319592_200121140049_R01C02                202      0.04160556
## 2   GSM5319603_3999356129_R02C02                306      0.06302625
## 3   GSM5319604_3999547012_R02C01               1156      0.23809916
## 4   GSM5319607_3999547012_R03C02                366      0.07538434
## 5   GSM5319609_3999547016_R02C01               2735      0.56332284
## 6   GSM5319613_3999547017_R02C01                460      0.09474534
## 7   GSM5319615_3999547017_R04C01                384      0.07909176
## 8   GSM5319616_3999547017_R06C01                674      0.13882252
```

As we can see from the table, the fifth 'GSM5319609_3999547016_R02C01' and third 'GSM5319604_3999547012_R02C01' samples display the higher percentage of failed probes.

6. Calculate raw beta and M values and plot the densities of mean methylation values, dividing the samples in CTRL and DIS (suggestion: subset the beta and M values matrixes in order to retain CTRL or DIS subjects and apply the function mean to the 2 subsets). Do you see any difference between the two groups?

Beta and M values represent the main metrics used to quantify the DNA methylation levels from Illumina microarray data.
The Beta values define the continous proportion of methylated molecules at a specific CpG locus; they range from 0 to 1. On the other hand, M values are calculated as the log2 ratio of the intensities of the methylated signal to the unmethylated signal ranging from -Inf to +Inf.

Use the functions `getBeta()` and `getM()` on the `MSet.raw` object, to retrieve beta and M values matrices containing the correspondent values for each probe of the samples.

```
beta <- getBeta(MSet.raw)
M <- getM(MSet.raw)
```

Inspect the outputs.

```
#summary(beta)
```

As we can see from the above summary table, the minimum for beta values is 0 while maximum corresponds to 1. We can also notice that there are also some `NA` values: these are the positions for which both the Methylation and Unmethylation values in the `MSet.raw` are equal to 0 (by default, in minfi the offset is set to 0).

```
#summary(M)
```

In the summary table for the M values we can check that the minimum for M values is -Inf (Methylation value=0, Unmethylation value>0), while the maximum is +Inf (Methylation value>0, Unmethylation value=0).
Also, we have also some `NA` s as we did in the summary table of the beta values.

Now we want to plot the density distribution of mean beta and M values dividing the samples in controls (CTRL) and diseases (DIS).
Create a data frame containing the samples IDs and the correspondent classification (CTRL and DIS) from the `RGset` object.

```
ctrl_dis_df <- data.frame(
  C_D = RGset@colData@listData[["Group"]],
  Samples = RGset@colData@rownames)
```

Subset the beta values matrices for both diseases and controls.

```
dis_Beta <- beta[, ctrl_dis_df$C_D=="DIS"]
ctrl_Beta <- beta[, ctrl_dis_df$C_D=="CTRL"]
```

Then, subset the M values matrices for both diseases and controls.

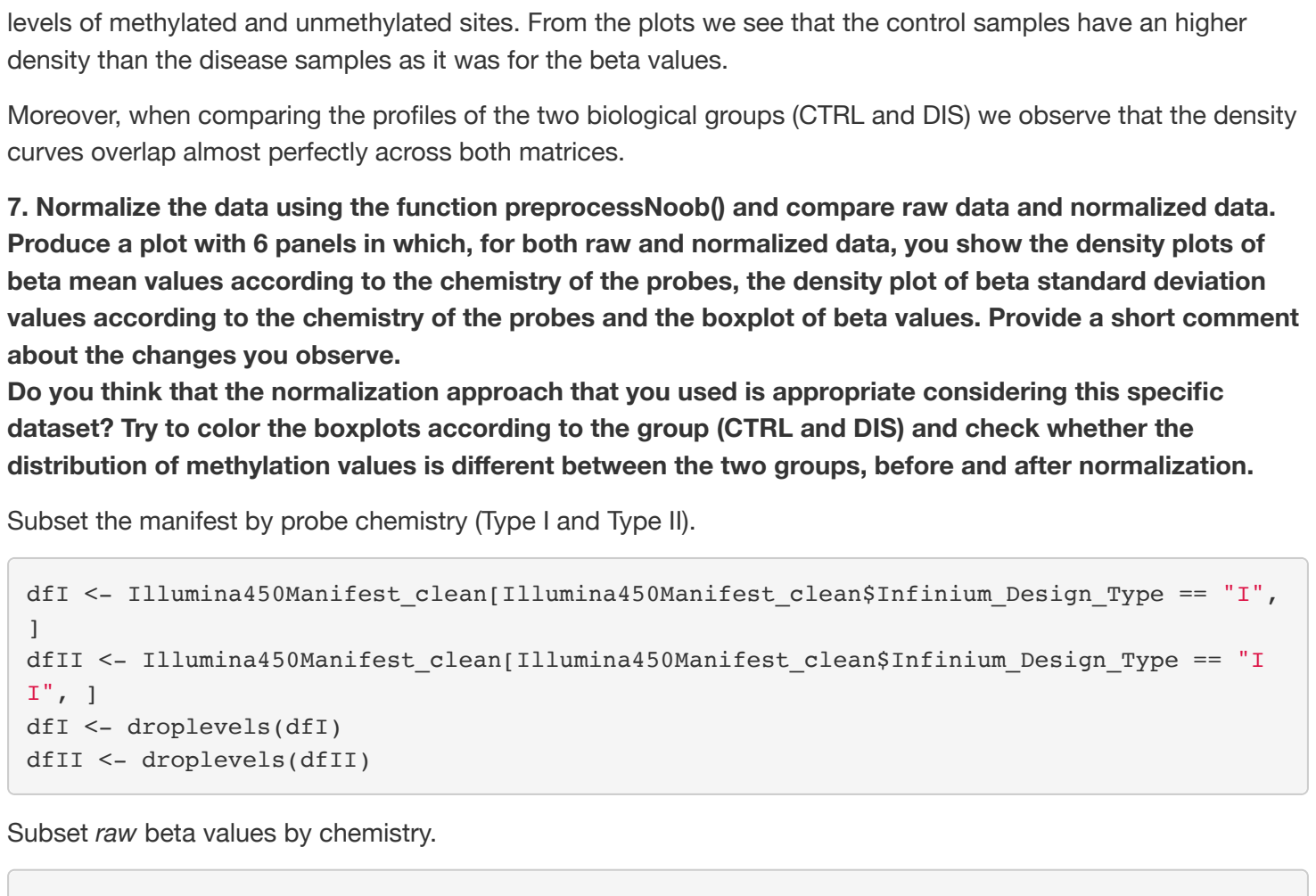
```
dis_M <- M[, ctrl_dis_df$C_D=="DIS"]
ctrl_M <- M[, ctrl_dis_df$C_D=="CTRL"]
```

Calculate mean values per probe across samples.

```
mean_ctrlBeta <- apply(ctrl_Beta, 1, mean, na.rm = TRUE)
mean_disBeta  <- apply(dis_Beta, 1, mean, na.rm = TRUE)

mean_ctrlM    <- apply(ctrl_M, 1, mean, na.rm = TRUE)
mean_disM     <- apply(dis_M, 1, mean, na.rm = TRUE)
```

Plot the density of mean Beta and M values.

```
par(mfrow = c(1, 2))

# Density of mean Beta values
plot(density(mean_ctrlBeta, na.rm = TRUE),
     main = "Density of Beta Values",
     col = "darkgoldenrod1", lwd = 2.5,
     xlab = "Mean Beta Values", ylab = "Density",
     xlim = c(-0.1,1.2), ylim = c(0,3.5))

lines(density(mean_disBeta, na.rm = TRUE), col = "lightgreen", lwd = 2.5)
legend("topright", legend = c("CTRL", "DIS"), col = c("darkgoldenrod1", "lightgreen"),
lwd = 2, cex = 0.8, seg.len = 1, bty = "n")

# Density of mean M values
plot(density(mean_ctrlM, na.rm = TRUE),
     main = "Density of M Values",
     col = "darkgoldenrod1", lwd = 2.5,
     xlab = "Mean M Values", ylab = "Density",
     ylim = c(0,0.30))
lines(density(mean_disM, na.rm = TRUE), col = "lightgreen", lwd = 2.5)
legend("topright", legend = c("CTRL", "DIS"), col = c("darkgoldenrod1", "lightgreen"),
lwd = 2, cex = 0.8, seg.len = 1, bty = "n")
```



```
#Reset plotting layout
par(mfrow = c(1, 1))
```

Density of Beta values The two peaks correspond to the levels of methylated (around 1.0) and unmethylated (around 0.0) samples, while the middle region corresponds to the heavy methylated state (equal amount of U and M). As we can see from the plots reporting the densities of the beta values, the control samples have a slightly higher density.

Density of M values Considering the plots for the M values, we have once again two peaks representing the levels of methylated and unmethylated sites. From the plots we see that the control samples have an higher density than the disease samples as it was for the beta values.

Moreover, when comparing the profiles of the two biological groups (CTRL and DIS) we observe that the density curves overlap almost perfectly across both matrices.

7. Normalize the data using the function preprocessNoob() and compare raw data and normalized data. Produce a plot with 6 panels in which, for both raw and normalized data, you show the density plots of beta mean values according to the chemistry of the probes, the density plot of beta standard deviation values according to the chemistry of the probes and the boxplot of beta values. Provide a short comment about the changes you observe. Do you think that the normalization approach that you used is appropriate considering this specific dataset? Try to color the boxplots according to the group (CTRL and DIS) and check whether the distribution of methylation values is different between the two groups, before and after normalization.

Subset the manifest by probe chemistry (Type I and Type II).

```
dfI <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_Type == "I",
]
dfII <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_Type == "I
I", ]
dfI <- droplevels(dfI)
dfII <- droplevels(dfII)
```

Subset *raw* beta values by chemistry.

```
beta_raw <- getBeta(MSet.raw)
beta_I_raw <- beta_raw[rownames(beta_raw) %in% dfI$IlmnID, ]
beta_II_raw <- beta_raw[rownames(beta_raw) %in% dfII$IlmnID, ]
```

Calculate mean and sd for *raw* Beta.

```
mean_beta_I_raw <- apply(beta_I_raw, 1, mean, na.rm = TRUE)
mean_beta_II_raw <- apply(beta_II_raw, 1, mean, na.rm = TRUE)
sd_beta_I_raw <- apply(beta_I_raw, 1, sd, na.rm = TRUE)
sd_beta_II_raw <- apply(beta_II_raw, 1, sd, na.rm = TRUE)
```

Normalize using the `preprocessNoob()` function.
Normal-exponential out-of-band (Noob) is a background correction method with dye-bias normalization for Illumina Infinium methylation arrays.

```
BiocManager::install("IlluminaHumanMethylation450kanno.ilmn12.hg19")

MSet.norm <- preprocessNoob(RGset)
beta_norm <- getBeta(MSet.norm)  # beta values of the normalized data
```

Subset *normalized* Beta by chemistry.

```
beta_I_norm <- beta_norm[rownames(beta_norm) %in% dfI$IlmnID, ]
beta_II_norm <- beta_norm[rownames(beta_norm) %in% dfII$IlmnID, ]
```

Calculate mean and sd for *normalized* Beta.

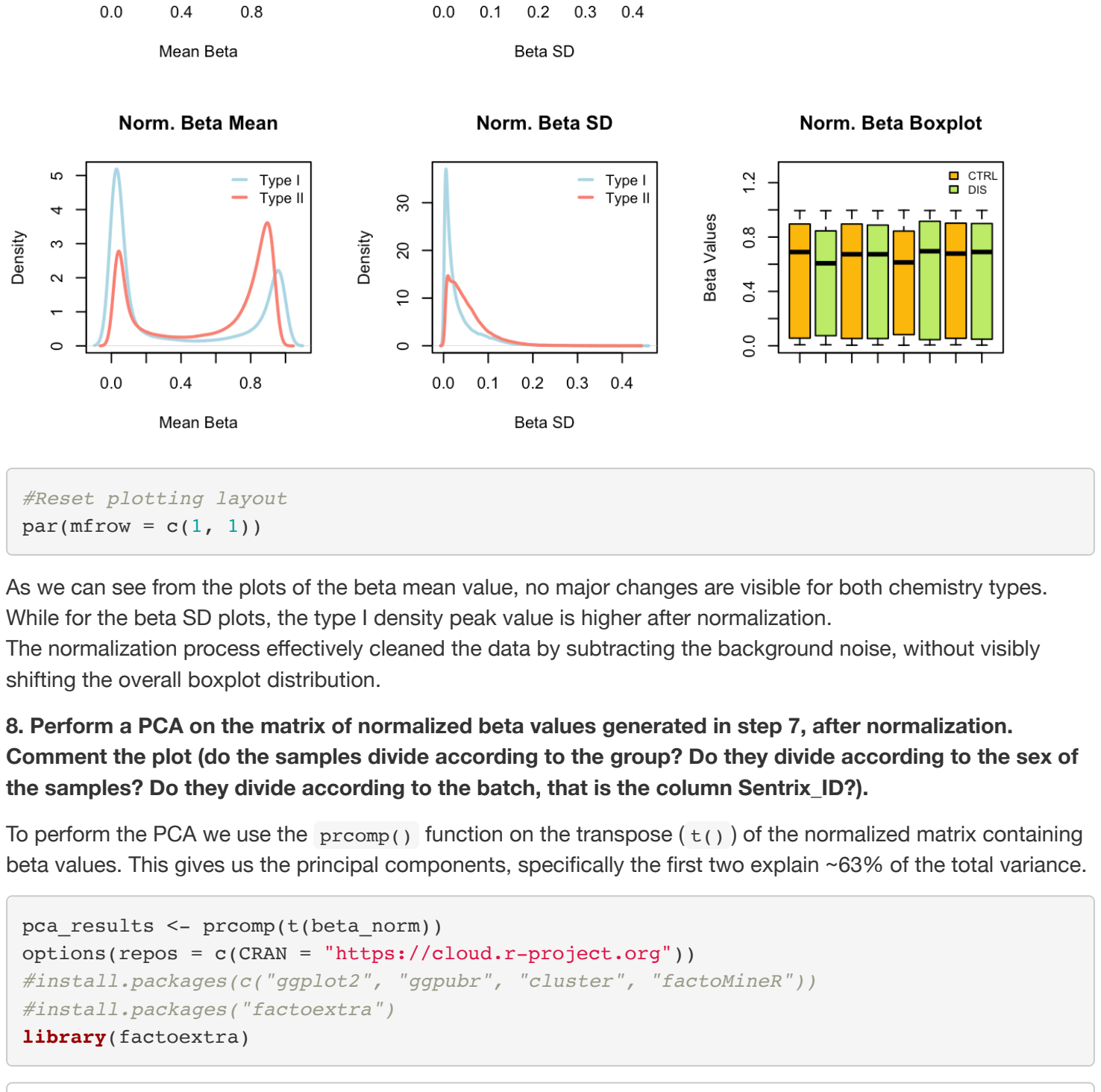
```
mean_beta_I_norm <- apply(beta_I_norm, 1, mean, na.rm = TRUE)
mean_beta_II_norm <- apply(beta_II_norm, 1, mean, na.rm = TRUE)
sd_beta_I_norm <- apply(beta_I_norm, 1, sd, na.rm = TRUE)
sd_beta_II_norm <- apply(beta_II_norm, 1, sd, na.rm = TRUE)
```

Prepare the colors for the groups to represent in the boxplots (the groups for raw beta were prepared in point 6).

```
dis_Beta_norm <- beta_norm[, ctrl_dis_df$C_D=="DIS"]
ctrl_Beta_norm <- beta_norm[, ctrl_dis_df$C_D=="CTRL"]
```

Plotting: 2 rows x 3 columns.

```
par(mfrow = c(2, 3))
# RAW DATA (first row)
# 1. Raw mean Beta densities
plot(density(mean_beta_I_raw, na.rm = TRUE), main = "Raw Beta Mean", col = "lightblue",
lwd = 2, xlab = "Mean Beta")
lines(density(mean_beta_II_raw, na.rm = TRUE), col = "salmon", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("lightblue", "salmon"), lwd
= 2,  cex = 0.9, seg.len = 1, bty = "n")

# 2. Raw sd Beta densities
plot(density(sd_beta_I_raw, na.rm = TRUE), main = "Raw Beta SD", col = "lightblue", lwd
= 2, xlab = "Beta SD")
lines(density(sd_beta_II_raw, na.rm = TRUE), col = "salmon", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("lightblue", "salmon"), lwd
= 2, cex = 0.9, seg.len = 1, bty = "n")

# 3. Raw Beta boxplot (colored by group)
boxplot(beta_raw, outline = FALSE, subset=c(ctrl_Beta, dis_Beta), col=c("darkgoldenrod
1", "darkolivegreen2"),
        main = "Raw Beta Boxplot", ylab = "Beta Values", names = rep("", ncol(beta_ra
w)), ylim = c(0, 1.3))
legend("topright", legend = c("CTRL", "DIS"), fill = c("darkgoldenrod1", "darkolivegree
n2"), bty = "n", cex = 0.7)



# NORMALIZED DATA (second row)
# 4. Normalized mean Beta densities
plot(density(mean_beta_I_norm, na.rm = TRUE), main = "Norm. Beta Mean", col = "lightblu
e", lwd = 2, xlab = "Mean Beta")
lines(density(mean_beta_II_norm, na.rm = TRUE), col = "salmon", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("lightblue", "salmon"), lwd
= 2, cex = 0.9, seg.len = 1, bty = "n")

# 5. Normalized sd Beta densities
plot(density(sd_beta_I_norm, na.rm = TRUE), main = "Norm. Beta SD", col = "lightblue",
lwd = 2, xlab = "Beta SD")
lines(density(sd_beta_II_norm, na.rm = TRUE), col = "salmon", lwd = 2)
legend("topright", legend = c("Type I", "Type II"), col = c("lightblue", "salmon"), lwd
= 2, cex = 0.9, seg.len = 1, bty = "n")

# 6. Normalized Beta boxplot (colored by group)
boxplot(beta_norm, outline = FALSE, subset=c(ctrl_Beta_norm, dis_Beta_norm), col=c("dar
kgoldenrod1", "darkolivegreen2"),
        main = "Norm. Beta Boxplot", ylab = "Beta Values", names = rep("", ncol(beta_no
rm)), ylim = c(0, 1.3))
legend("topright", legend = c("CTRL", "DIS"), fill = c("darkgoldenrod1", "darkolivegree
n2"), bty = "n", cex = 0.7)
```



```
#Reset plotting layout
par(mfrow = c(1, 1))
```

As we can see from the plots of the beta mean value, no major changes are visible for both chemistry types. While for the beta SD plots, the type I density peak value is higher after normalization.
The normalization process effectively cleaned the data by subtracting the background noise, without visibly shifting the overall boxplot distribution.

8. Perform a PCA on the matrix of normalized beta values generated in step 7, after normalization. Comment the plot (do the samples divide according to the group? Do they divide according to the sex of the samples? Do they divide according to the batch, that is the column Sentrix_ID?).

To perform the PCA we use the `prcomp()` function on the transpose (`t()`) of the normalized matrix containing beta values. This gives us the principal components, specifically the first two explain ~63% of the total variance.

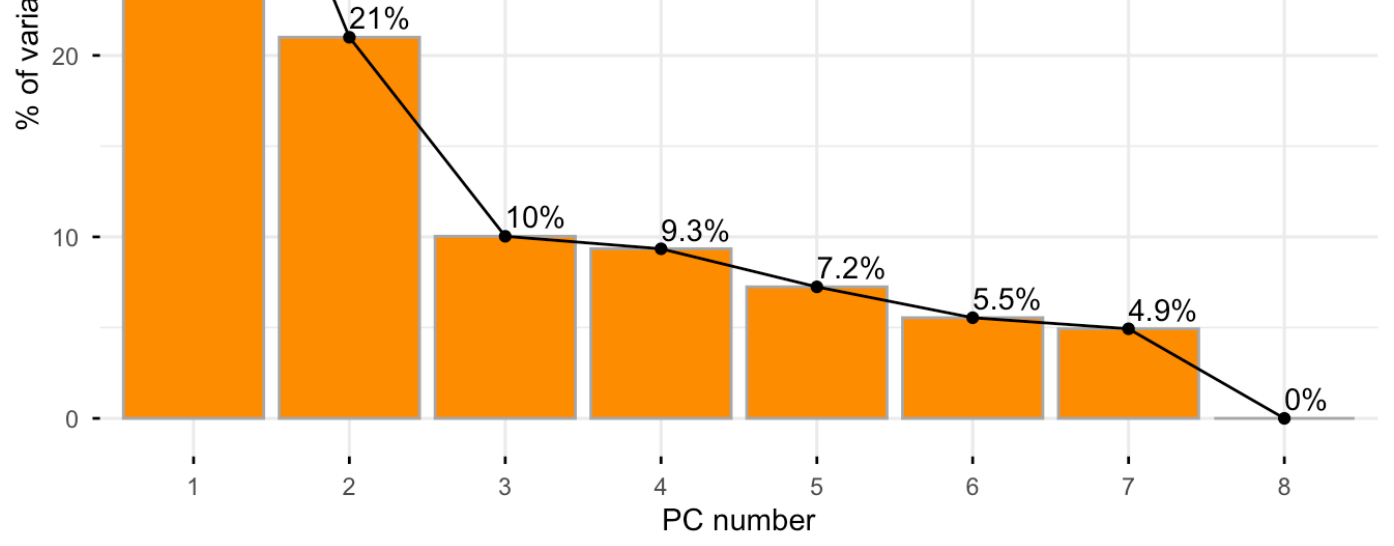
```
pca_results <- prcomp(t(beta_norm))
options(repos = c(CRAN = "https://cloud.r-project.org"))
#install.packages(c("ggplot2", "ggpubr", "cluster", "factoMineR"))
#install.packages("factoextra")
library(factoextra)
```

```
## Loading required package: ggplot2
```

```
## Welcome to factoextra!
```

```
## Want to learn more? See two factoextra-related books at https://www.datanovia.com/e
n/product/practical-guide-to-principal-component-methods-in-r/
```

```
fviz_eig(pca_results, addlabels = TRUE, xlab = 'PC number', ylab = '% of variance', bar
fill = "darkorange", barcolor = "darkgrey")
```

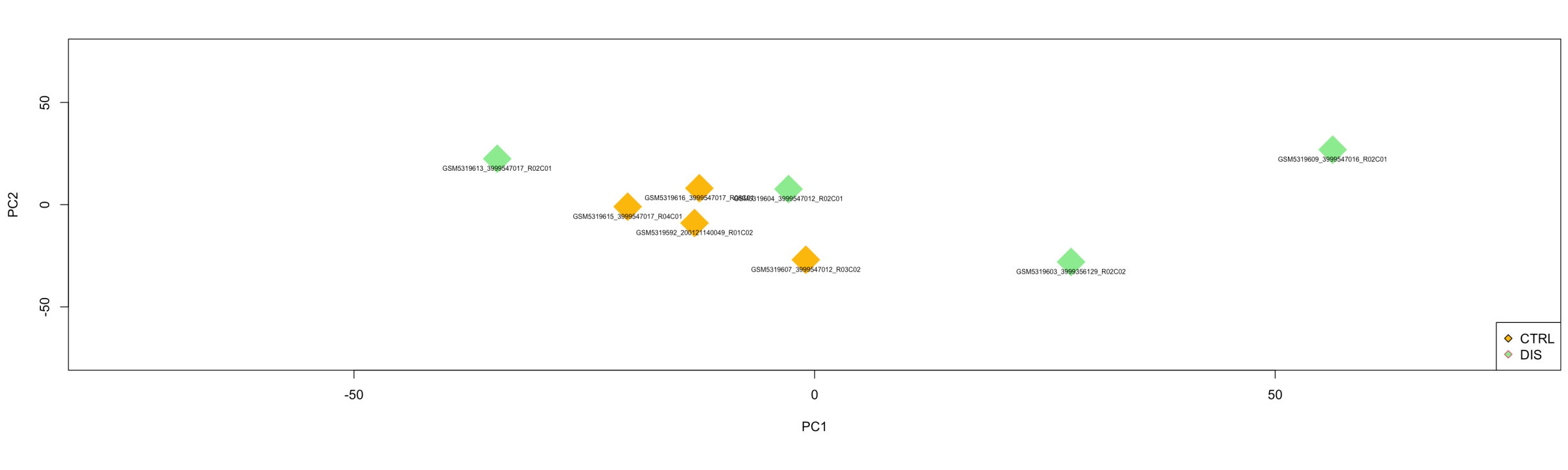


The Scree Plot illustrates the distribution of variance across the principal components. The first two principal components account for approximately 63% of the total variance, with a distinct ‘elbow’ observed after PC2. This indicates that the primary biological variance within the dataset is effectively captured by the first two dimensions, justifying the use of PC1 and PC2 for our 2D visualization and downstream clustering analysis.

We color the samples according to different features to visually check the degrees of separation among them. Specifically we are going to color them based on:

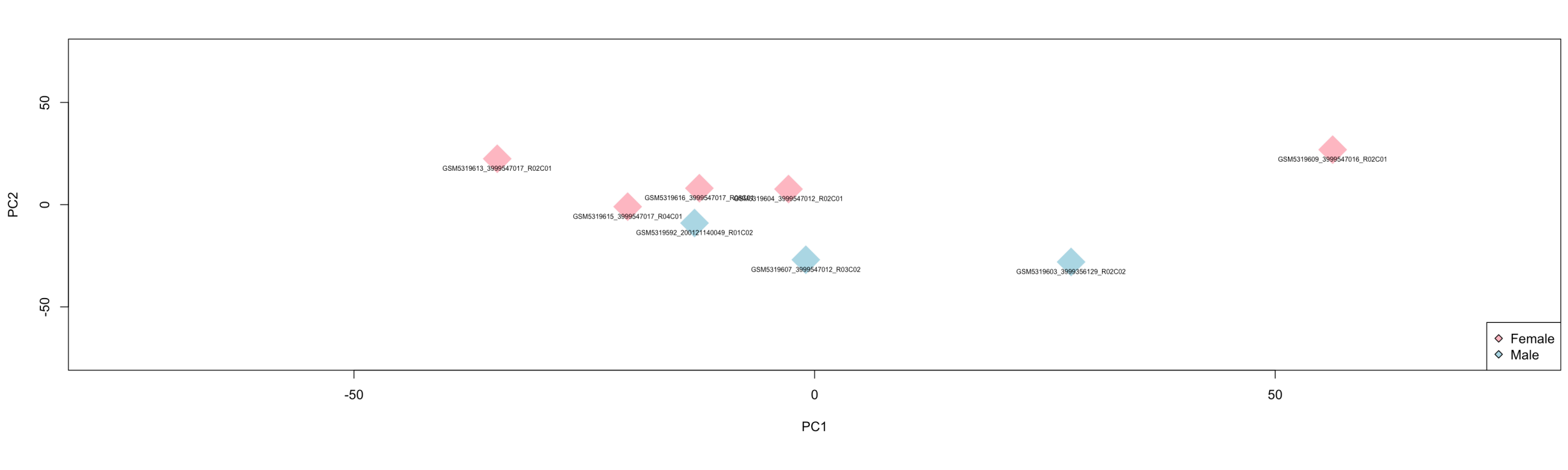
- Group (“CTRL” VS “DIS”)

```
#plot(pca_results$x[,1], pca_results$x[,2],cex=1,pch=2)
targets$Group <- as.factor(targets$Group)
#levels(targets$Group)

group_colors <- c("darkgoldenrod1", "lightgreen")
plot(pca_results$x[,1], pca_results$x[,2],cex=4,pch=23, col="white",bg = group_colors[a
s.numeric(targets$Group)],xlab="PC1",ylab="PC2",xlim=c(-75,75),ylim=c(-75,75))
text(pca_results$x[,1], pca_results$x[,2],labels=rownames(pca_results$x),cex=0.5,pos=1)
legend("bottomright",legend=levels(targets$Group),col=c(1:nlevels(targets$Group)),pt.bg
=group_colors,pch=23)
```



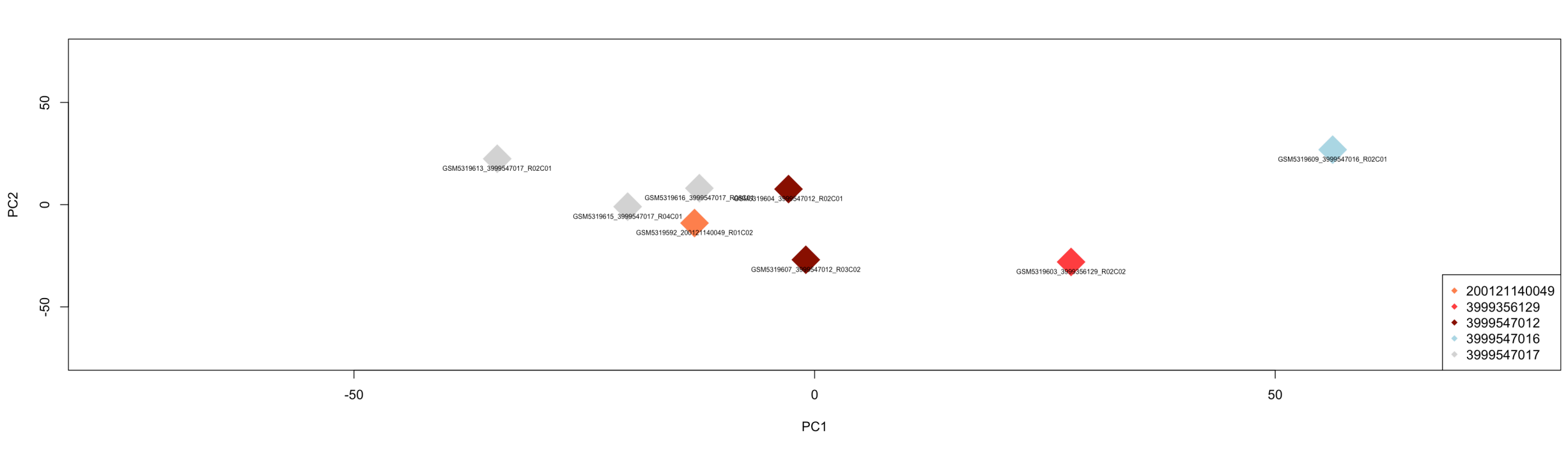
- Sex (“Female” VS “Male”)

```
targets$Sex <- as.factor(targets$Sex)
#levels(targets$Sex)
sex_colors <- c("lightpink", "lightblue")
plot(pca_results$x[,1], pca_results$x[,2],cex=4,pch=23,col="white",bg = sex_colors[as.n
umeric(targets$Sex)],xlab="PC1",ylab="PC2",xlim=c(-75,75),ylim=c(-75,75))
text(pca_results$x[,1], pca_results$x[,2],labels=rownames(pca_results$x),cex=0.5,pos=1)
legend("bottomright",legend=levels(targets$Sex),pt.bg = sex_colors,pch=23)
```



- Batch (different slides of the experiments)

```
targets$Slide <- as.factor(targets$Slide)

#in order to assign the colors i search for the unique slides
#unique(targets$Slide)
#table(targets$Slide)

#i have 5 unique slides
slide_colors <- c("coral", "brown1", "darkred", "lightblue", "lightgrey")
plot(pca_results$x[,1], pca_results$x[,2],cex=4,pch=23,col="white",bg = slide_colors[a
s.numeric(targets$Slide)],xlab = "PC1",ylab="PC2",xlim=c(-75,75),ylim=c(-75,75))
text(pca_results$x[,1], pca_results$x[,2],labels=rownames(pca_results$x),cex=0.5,pos=1)
legend("bottomright",legend=levels(targets$Slide),col="white",pt.bg=slide_colors,pch=2
3)
```



PCA visualization reveals that the primary source of variance within the dataset is associated with the technical batch effect (Slide), rather than the biological variables of interest (Group) or demographic factors (Sex). The clear stratification by slide indicates that technical variation is the dominant driver of sample clustering, which may potentially confound downstream differential methylation analysis

9. Using the matrix of normalized beta values generated in step 7, identify differentially methylated probes between group CTRL and group DIS using the function assigned to each group.

```
# This function takes a single row (probe) of beta values and compares them by group
my_mannwhitney_test <- function(x) {
  # We use exact = FALSE because methylation data often contains ties
  wilcox <- wilcox.test(x ~ targets$Group, exact = FALSE)
  return(wilcox$p.value)
}

# Apply the test to every row (probe) in your normalized matrix
pValues_wilcox <- apply(beta_norm, 1, my_mannwhitney_test)

# Combine your beta values and the newly calculated p-values
test_df <- data.frame(beta_norm, pValues_wilcox)

#keep only those with p-value <= 0.05
diff_meth_probes <- test_df[test_df$pValues_wilcox <= 0.05, ]

# Check how many you found
nrow(diff_meth_probes)
```

```
## [1] 11052
```

The results obtained via the Wilcoxon-Mann-Whitney test with a threshold of p≤0.05 should be interpreted exclusively as nominally significant; so 11052 represent nominal significant results.
It is essential to emphasize that in the context of high-throughput genomic analyses—such as the Illumina 450k microarray—this approach is inherently misleading due to the ‘multiple testing problem’. When performing over 450,000 simultaneous statistical tests, the application of a standard significance threshold leads inevitably to a high rate of false positives: by pure random chance, approximately 5% of all tests will appear significant even in the absence of a genuine biological effect.
Furthermore, the use of the Wilcoxon-Mann-Whitney test in this context introduces a specific technical constraint. As a rank-based non-parametric test, its application to methylation data—which frequently contains numerous identical values (or ‘ties’)—results in a discretization of p-values. This phenomenon manifests as a horizontal stratification of results, making the distribution of nominal p-values less continuous and more prone to artificial clustering.
Consequently, the number of probes identified as ‘significant’ at the nominal level drastically overestimates the true biological scope of the phenomenon. To isolate robust signals, it is necessary to apply rigorous statistical corrections, such as the Benjamini-Hochberg (BH) procedure for controlling the False Discovery Rate (FDR) or the Bonferroni correction. These methods penalize the p-value based on the total number of tests performed, effectively filtering out the background noise generated by the multiple testing burden

10. Apply multiple test correction and set a significant threshold of 0.05. How many probes do you identify as differentially methylated considering nominal pValues? How many after Bonferroni correction? How many after BH correction?

```
corrected_pValues_BH <- p.adjust(test_df$pValues_wilcox,method="BH")
corrected_pValues_Bonf <-p.adjust(test_df$pValues_wilcox,method="bonferroni")
sum(corrected_pValues_BH <= 0.05)
```

```
## [1] 0
```

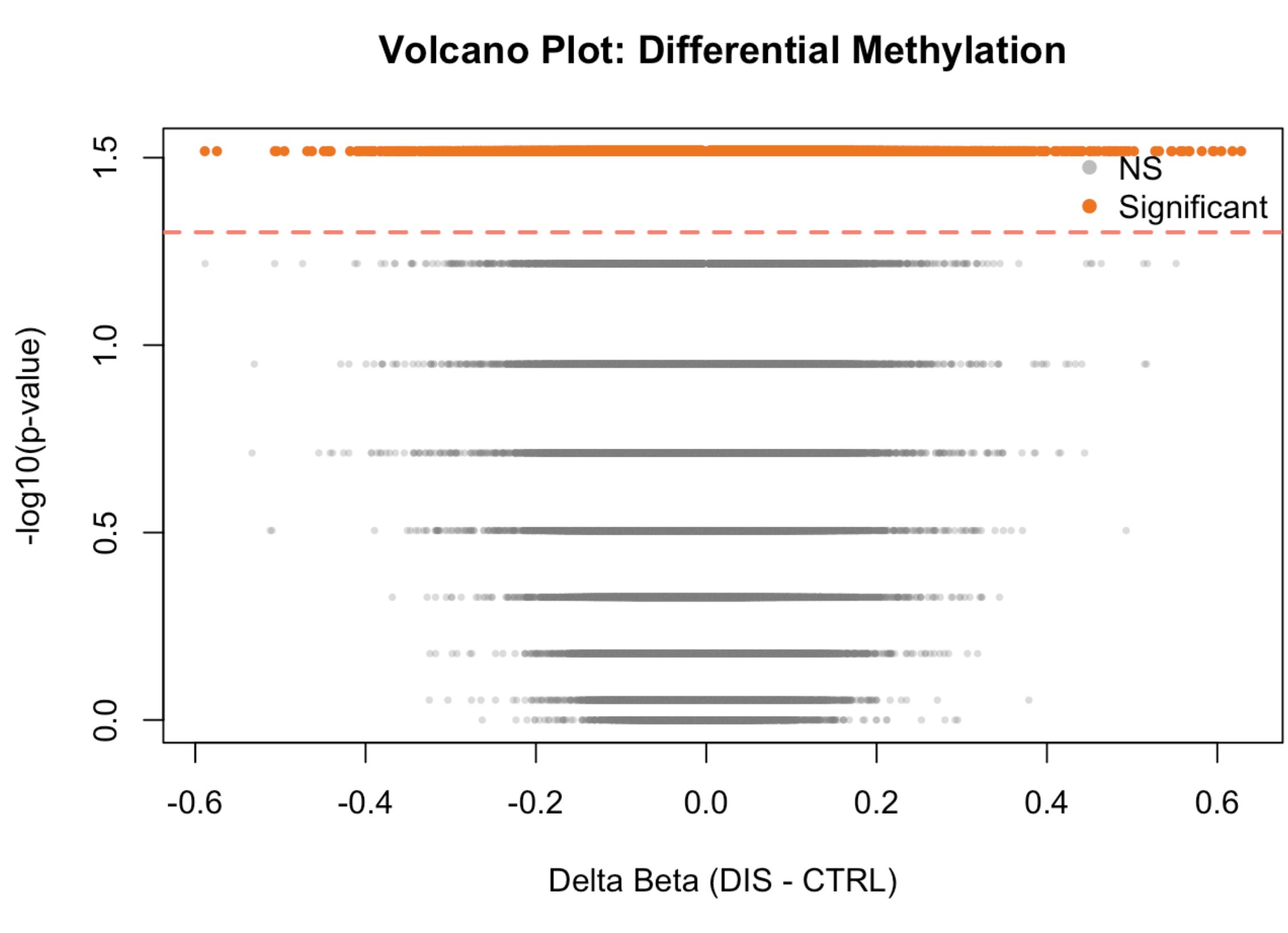
```
sum(corrected_pValues_Bonf <= 0.05)
```

```
## [1] 0
```

The identification of zero probes achieving significance after Bonferroni and Benjamini-Hochberg (BH) corrections indicates that the differences between the *CTRL* and *DIS* groups are not robust enough to withstand the stringent penalties imposed by multiple testing. As previously suggested by our PCA results, this outcome is likely due to the presence of dominant confounding factors—specifically the experimental Slide (batch effect)—which appear to account for a larger proportion of the observed variance than the experimental grouping itself.
To better evaluate the impact of these corrections and visualize the underlying distribution of our data, we generated boxplots comparing nominal p-values against those adjusted by BH and Bonferroni procedures. This visualization highlights the ‘shift’ in statistical stringency: while many probes appear nominally significant, the adjustment process effectively pushes the vast majority of these values above the significance threshold, confirming that the initial signals were likely artifacts of random noise or technical variance rather than genuine biological differences

11.Produce a volcano plot and a Manhattan plot of the results of differential methylation analysis

```
# Calculate means for each group
ctrl_indices <- which(targets$Group == "CTRL")
dis_indices <- which(targets$Group == "DIS")

mean_ctrl <- rowMeans(beta_norm[, ctrl_indices], na.rm = TRUE)
mean_dis <- rowMeans(beta_norm[, dis_indices], na.rm = TRUE)
delta_beta <- mean_dis - mean_ctrl

toVolcPlot <- data.frame(delta = delta_beta, logP = -log10(test_df$pValues_wilcox))

# 3. Plotting
plot(toVolcPlot$delta, toVolcPlot$logP,
     pch = 16, cex = 0.5,
     col = rgb(0.5, 0.5, 0.5, 0.3),
     xlab = "Delta Beta (DIS - CTRL)",
     ylab = "-log10(p-value)",
     main = "Volcano Plot: Differential Methylation")

abline(h = -log10(0.05), col = "salmon", lwd = 2, lty = 2)

sig_idx <- which(toVolcPlot$logP > -log10(0.05))
points(toVolcPlot$delta[sig_idx], toVolcPlot$logP[sig_idx],
       pch = 16, cex = 0.7, col = "chocolate2")

# 6. Legenda
legend("topright", legend = c("NS", "Significant"),
       col = c("gray", "chocolate2"), pch = 16, bty = "n")
```



The volcano plot displays nominal p-values against delta beta values. The observed horizontal stratification of p-values is a direct consequence of the ties present in methylation beta values when using the Wilcoxon rank-sum test . Because this test is fundamentally rank-based, it calculates significance by comparing the relative order of observations rather than their absolute magnitude. In methylation datasets, where many probes frequently share identical beta values—a phenomenon known as ‘ties’—the test is mathematically constrained to return a discrete, finite set of possible p-values.
This discretization creates a ‘banding’ effect on the Y-axis, which limits the resolution of the Volcano plot . When probes are ranked, the test cannot distinguish between variations in magnitude that fall within the same rank-sum distribution, effectively ‘binning’ results into specific horizontal strata. Upon applying multiple testing correction (Benjamini-Hochberg), these discrete levels often fail to meet genome-wide significance thresholds. This suggests that the observed variability in the data is more strongly associated with batch effects (e.g., Slide) than with the experimental grouping, consistent with observations from our PCA analysis .

Manhattan plot

```
library(IlluminaHumanMethylation450kanno.ilmn12.hg19)
anno <- getAnnotation(IlluminaHumanMethylation450kanno.ilmn12.hg19)

# We need 'chr' and 'pos' (base position)
anno_df <- as.data.frame(anno[, c("chr", "pos")])

manhattan_data <- data.frame(
    SNP = rownames(test_df),
    P = test_df$pValues_wilcox,
    CHR = anno_df[rownames(test_df), "chr"],
    BP = anno_df[rownames(test_df), "pos"]
)

manhattan_data$CHR <- as.numeric(gsub("chr", "", manhattan_data$CHR))
```

```
## Warning: NAs introduced by coercion
```

```
manhattan_data <- manhattan_data[!is.na(manhattan_data$CHR), ]
manhattan_data <- manhattan_data[!is.na(manhattan_data$P), ] #

library(qqman)
```

```
##
```

```
## For example usage please run: vignette('qqman')
```

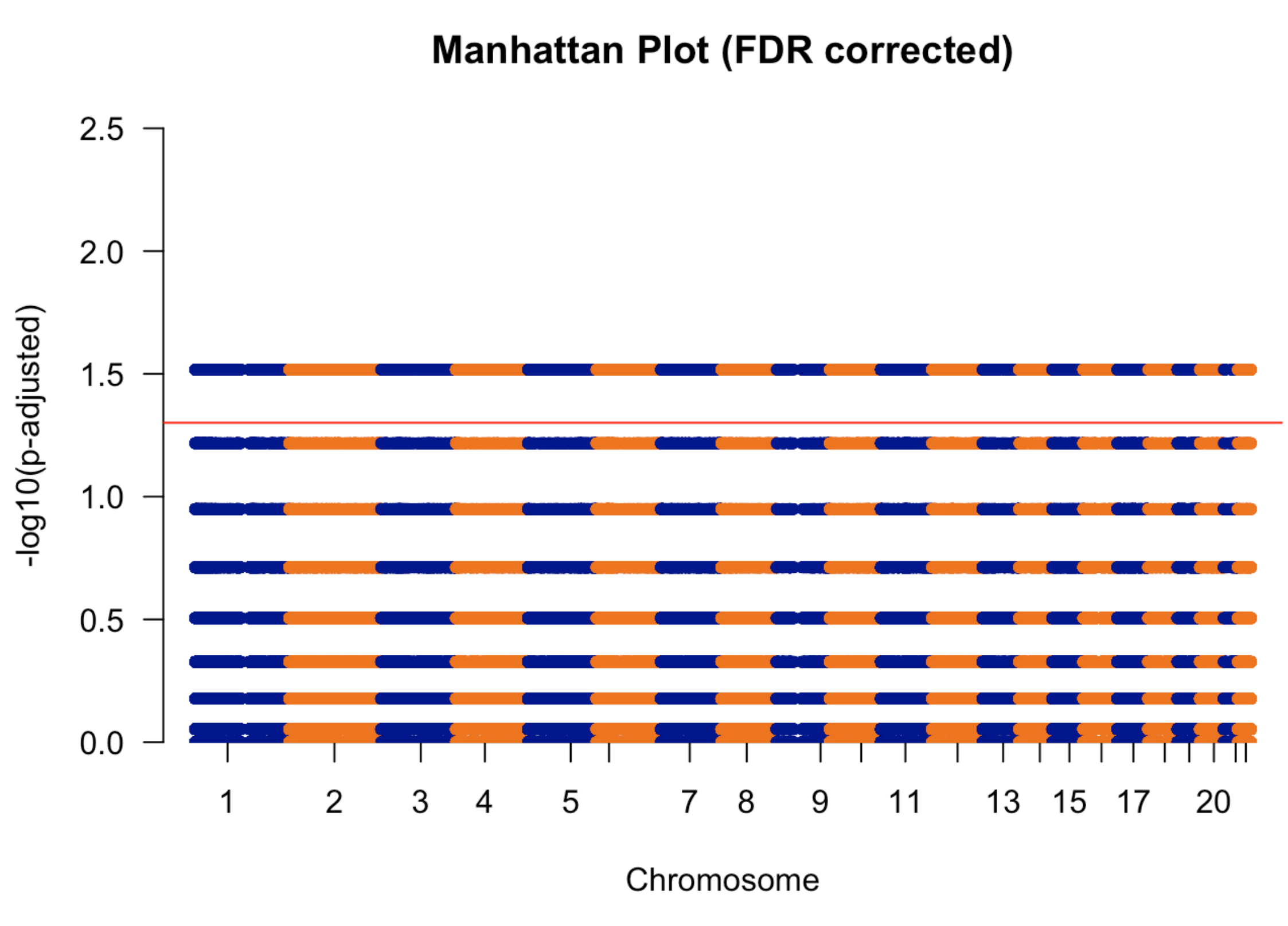
```
##
```

```
## Citation appreciated but not required:
```

```
## Turner, (2018). qqman: an R package for visualizing GWAS results using Q-Q and manha
ttan plots. Journal of Open Source Software, 3(25), 731, https://doi.org/10.21105/joss.
00731.
```

```
##
```

```
threshold_fdr <- -log10(0.05)

manhattan(manhattan_data,
          main = "Manhattan Plot (FDR corrected)",
          col = c("blue4", "chocolate2"),
          suggestiveline = FALSE,
          genomewideline = threshold_fdr,
          ylim = c(0, max(-log10(manhattan_data$P), 2) + 0.5),
          ylab = "-log10(p-adjusted)")
```



For the visualization of genomic results, we chose to represent the nominal p-values obtained via the Wilcoxon-Mann-Whitney test. This choice is motivated by the desire to observe the distribution of the ‘raw’ signal across the genome, as the application of extremely stringent multiple-testing corrections (such as Bonferroni or Benjamini-Hochberg) in this specific dataset resulted in the elimination of all significant signals, making it impossible to identify potential regions of interest. It is important to note that the distribution of p-values in this Manhattan plot exhibits a marked horizontal stratification. This phenomenon is a technical quantization artifact due to the non-parametric nature of the Wilcoxon test: being rank-based.
Consequently, the –log10(p) values map to fixed levels, generating the observed stratification. While we acknowledge that these bands represent nominal significance thresholds, they allow for the analysis of signal consistency on a chromosomal scale, highlighting how, in the absence of conservative correction, the distribution does not show isolated genomic peaks but rather a distributed systemic response

12. Heatmap

```
#install.packages("gplots")


library(gplots)
```

```
##
## ---------------------
## gplots 3.3.0 loaded:
##   * Use citation('gplots') for citation info.
##   * Homepage: https://talgalili.github.io/gplots/
##   * Report issues: https://github.com/talgalili/gplots/issues
##   * Ask questions: https://stackoverflow.com/questions/tagged/gplots
##   * Suppress this message with: suppressPackageStartupMessages(library(gplots))
## ---------------------
```

```
##
## Attaching package: 'gplots'
```

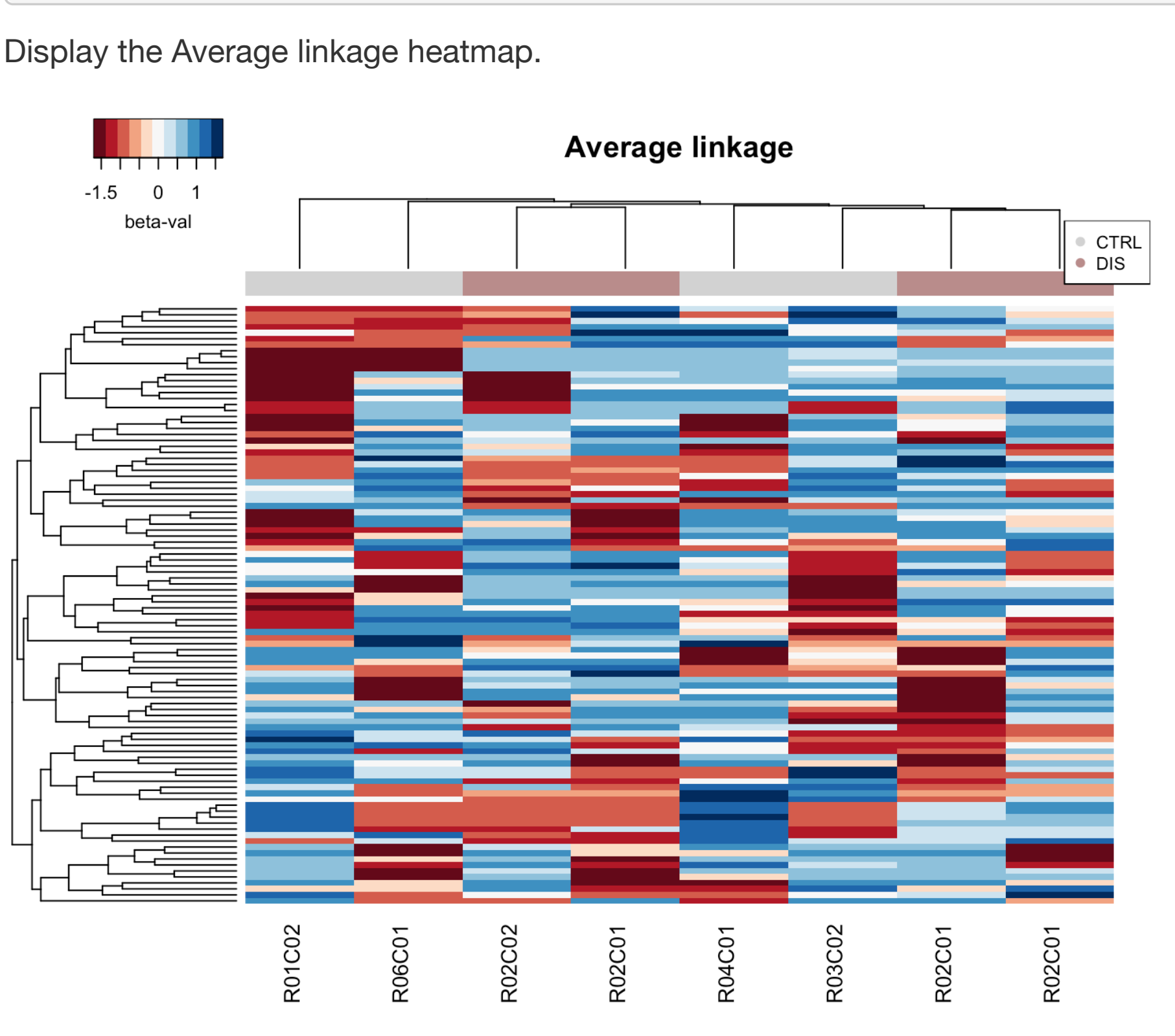
```
## The following object is masked from 'package:IRanges':
##
##     space
```

```
## The following object is masked from 'package:S4Vectors':
##
##     space
```

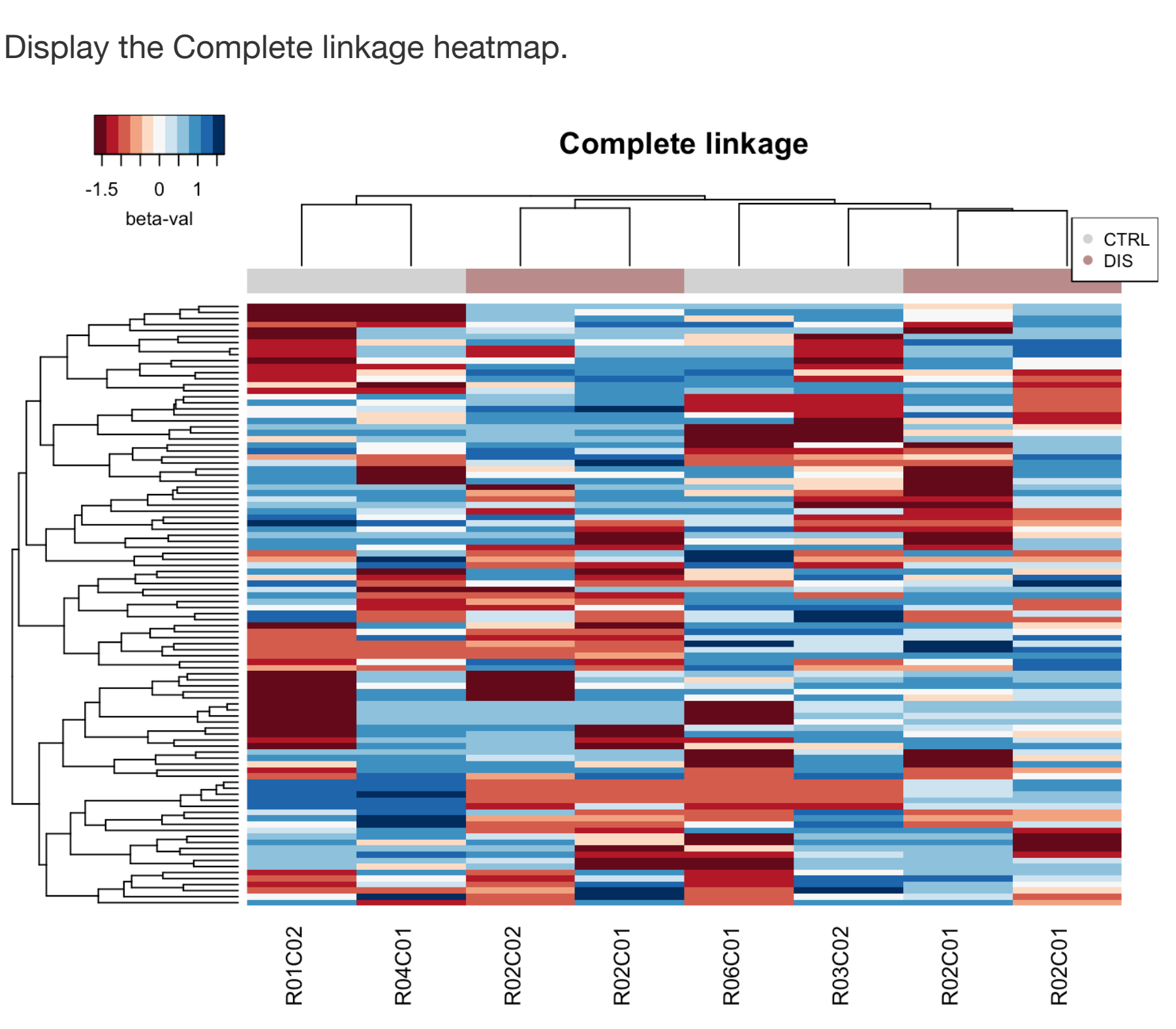
```
## The following object is masked from 'package:stats':
##
##     lowess
```

```
# Sort by variance to see the most "informative" probes
vars <- apply(beta_norm, 1, var)
top_probes <- order(vars, decreasing = TRUE)[1:100]
input_heatmap <- as.matrix(beta_norm[top_probes, ])
group_colors_map <- c("CTRL" = "lightgrey", "DIS" = "rosybrown")
```

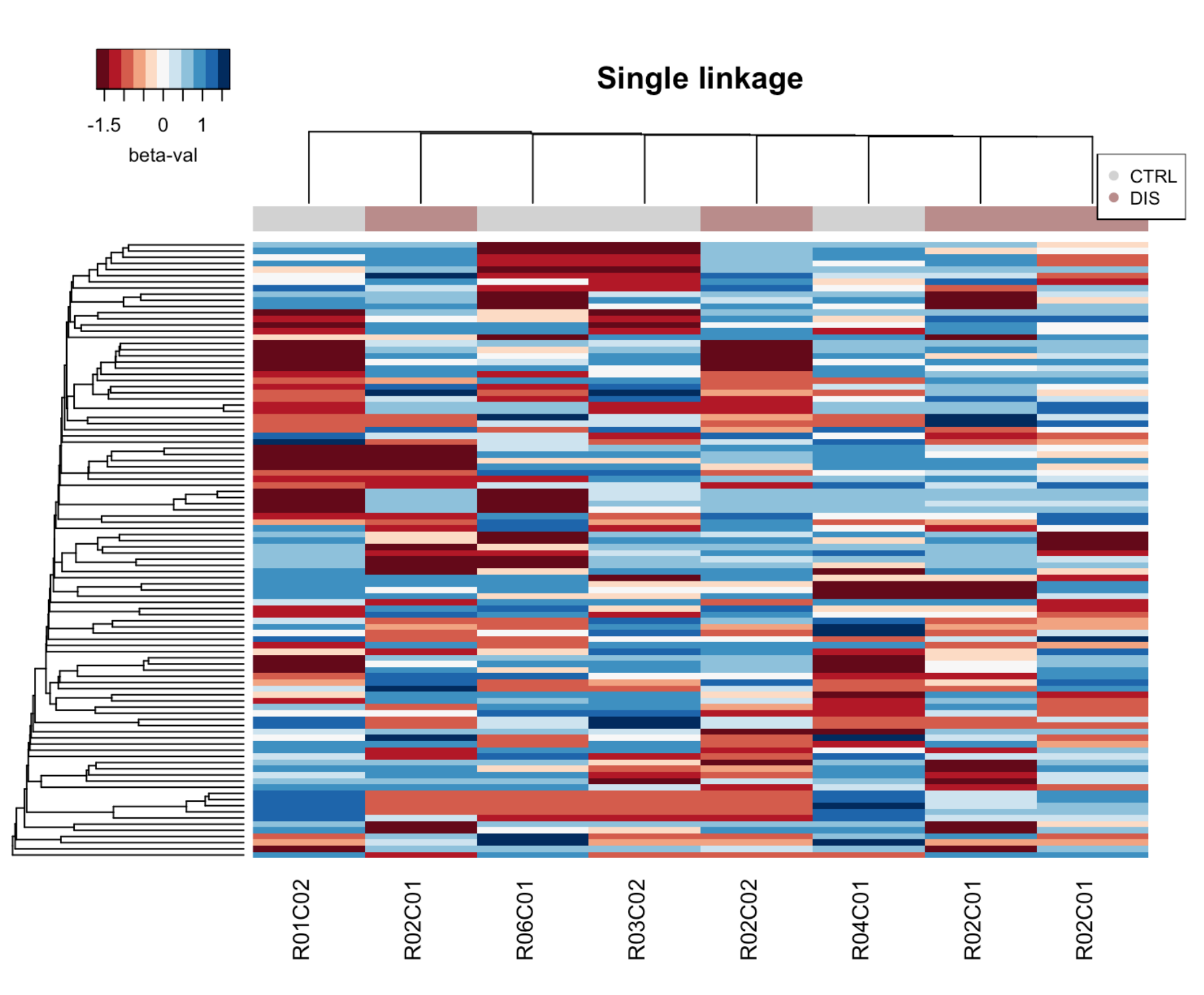
Display the Average linkage heatmap.



Display the Complete linkage heatmap.



Display the Single linkage heatmap.



The analysis of the heatmaps, utilizing Average, Complete, and Single linkage methods, reveals distinct structural hierarchies within the dataset. Average and Complete linkage methods demonstrate more defined clustering patterns; Average linkage creates a balanced, coherent grouping by minimizing overall pairwise distances, while Complete linkage identifies compact, distinct “clumps” by focusing on the most distant cluster members. In contrast, the Single linkage method exhibits a “chaining” effect, characterized by sequential connectivity that highlights local relationships rather than rigid group boundaries.
Crucially, the fact that these algorithms successfully group samples into clear hierarchies—yet fail to align with the experimental ‘CTRL’ and ‘DIS’ labels—further confirms that the primary variation driving these clusters is likely the technical batch effect (Slide) rather than a genuine biological signal. Collectively, these visualizations suggest that the clustering algorithms are effectively organizing the systemic technical noise into the clear structural patterns observed across the heatmaps.

CONCLUSION In summary, the differential methylation analysis did not yield any statistically significant results after applying Bonferroni and Benjamini-Hochberg corrections for multiple testing. This outcome is consistent with our exploratory data analysis: the PCA plots clearly reveal that the primary source of variance in the data is associated with the ‘Slide’ factor rather than the experimental biological conditions. Together, the subsequent lack of significant statistical findings indicate that technical batch effects are likely masking the underlying biological signal. Consequently, no reliable biological differences could be identified without first addressing these systemic technical artifacts through further normalization or batch-effect correction.
Moreover, the Volcano and Manhattan plots exhibit distinct ‘striping’ patterns, which reflect the systematic impact of the rank-based Mann-Whitney test applied to the data, rather than the distribution of true biological significance. Finally, the heatmaps confirm this by showing that samples cluster strictly according to technical batch effects rather than their ‘CTRL’ or ‘DIS’ labels.